CONCLUSION:

Bubble sort:
We successfully executed the sorting of the unsorted array using bubble sort. The algorithm repeatedly compares and swaps the elements until all the elements are sorted or no possible swaps since the execution time is comparatively higher than other sorting methods this is not efficient on large arrays. Therefore this method is only useful in sorting the small array or datasets.

Insertion sort:
We successfully executed the sorting of the unsorted array using insertion sort. The algorithm inserts the elements into the proper position with in the array by swapping. It is not efficient on the large unsorted array but very efficient on small and sorted array due to its long execution time. Therefore this method is impractical on large datasets or real-word data due to its large memory.

Selection sort:
We successfully implemented selection sort on an unsorted array. This algorithm selects the smallest element from and unsorted array and places it in the correct position. Even it has less swaps than bubble sort still the execution time is longer. Therefore this method is only suitable for small array or small datasets.

Merge sort:
We successfully executed the merge sort on an unsorted array. This algorithm uses divide-and-conquer method for sorting ,here the large array is divided into smaller sub-array and sorts them, and merges them into fully sorted array. It provides consistent performance for all datasets . since it requires additional memory for merging , this method is ideal for large datasets.

Quick sort:
We successfully executed the quick sort on an unsorted array. This algorithm selects an pivot element for partitioning the array into smaller sub arrays. This algorithm as the fastest execution time among all the sorting algorithms. In the worst case the execution time is long due to poor piviot selection. Therefore quick sort provides minimal execution

time and widely used for sorting large datasets or large arrays due to its high efficiency.